

CO3-AT3-Debugging and Optimization

View Only

[← Back](#)

QUESTIONS

Q1ATM Withdrawal – Exception
Handling Debugging
10 marks**Q2**Hospital Registration –
NullPointerException
Debugging
10 marks**Q3**E-Commerce Product Search –
Array Exception Debugging
10 marks**Q4**Bank Account – Race Condition
Debugging
10 marks**Q5**Railway Reservation –
Synchronization Debugging

Q4 Bank Account – Race Condition Debugging

10 marks

A bank account contains ₹10,000. Two ATM threads simultaneously attempt to withdraw ₹7,000 and ₹6,000.

The following program contains a race condition.

Task:

1. Identify the race condition.
2. Identify the critical section.
3. Explain why both threads may pass the balance check.
4. Modify the complete program using synchronized.
5. Ensure that the balance can never become negative.
6. Submit the COMPLETE corrected Java program.

Faulty Program:

```
class BankAccount {  
  
    int balance = 10000;  
  
    void withdraw(int amount) {  
  
        if (balance >= amount) {
```

10 marks

5/5 answered

```
System.out.println(
Thread.currentThread().getName()
+ " withdrawing " + amount);

try {
Thread.sleep(100);
}
catch (InterruptedException e) {
System.out.println("Interrupted");
}

balance = balance - amount;

System.out.println(
Thread.currentThread().getName()
+ " withdrawal successful");
}
else {
System.out.println(
Thread.currentThread().getName()
+ " insufficient balance");
}
}
}

class ATM extends Thread {

BankAccount account;
int amount;
```

```
ATM(BankAccount account, int amount) {  
    this.account = account;  
    this.amount = amount;  
}  
  
public void run() {  
    account.withdraw(amount);  
}  
}  
  
class BankApplication {  
  
    public static void main(String[] args)  
        throws InterruptedException {  
  
        BankAccount account = new BankAccount();  
  
        ATM atm1 = new ATM(account, 7000);  
        ATM atm2 = new ATM(account, 6000);  
  
        atm1.setName("ATM-1");  
        atm2.setName("ATM-2");  
  
        atm1.start();  
        atm2.start();  
  
        atm1.join();  
        atm2.join();  
  
        System.out.println(  
            "Final Balance = " + account.balance);  
        }  
    }  
}
```

Correct the program so that only one withdrawal succeeds.

🔧 Debugging Task: Find and fix the bugs in the code shown above. Write your corrected code in the editor below.

Your Fixed Code

```
1 class BankAccount{
2     int balance=10000;
3     synchronized void withdraw(int amount){
4         if(balance>=amount){
5             System.out.println(Thread.currentThread
6                 .getName()+" withdrawing "+amount);
7             Thread.sleep(100);
8         }catch(InterruptedException e){
9             System.out.println(" Interrupted");
10        }
11        balance=balance-amount;
12        System.out.println(Thread.currentThread
13            .getName()+" withdrawal successful");
14    }else{
15        System.out.println(Thread.currentThread
16            .getName()+" insufficient balance");
17    }
18 }
19 public static void main(String[] args)throws In
20     BankAccount account=new BankAccount();
21     ATM atm1=new ATM(account,7000);
22     ATM atm2=new ATM(account,6000);
23     atm1.setName("ATM-1");
24     atm2.setName("ATM-2");
25     atm1.start();
26     atm2.start();
```

Test Input

stdin input (optional)

Output

ATM-1 withdrawing 7000
ATM-1 withdrawal
successful
ATM-2 insufficient
balance
Final Balance = 3000

🕒 Pending manual review

```
25         atm1.join();
26         atm2.join();
27         System.out.println("Final Balance = "+accou
28     }
29 }
30 class ATM extends Thread{
31     BankAccount account;
32     int amount;
33     ATM(BankAccount account,int amount){
34         this.account=account;
35         this.amount=amount;
36     }
37     public void run(){
38         account.withdraw(amount);
39     }
40 }
```



✓ Submitted

© 2026 **SIMATS Online Lab**. All rights reserved.